Lossless Compression

Zhicheng Li

1 Overview

2 Context Modeling and Probability Estimation

Prediction by partial matching (PPM) provides probability estimates. An arithmetic coder uses these estimates to produce a compressed bitstream. PPM estimates the probability of the next symbol using previously observed symbols as a context. When a longer context cannot predict the next symbol, the model backs off to a shorter one [1].

2.1 Context Representation

Our implementation reads the input file as a sequence of bytes, treating each byte as a symbol from the alphabet $\mathcal{B} = \{0, \ldots, 255\}$. Let $x_t \in \mathcal{B}$ denote the t-th input symbol, starting at $t = 1$. The *history* h stores the most recent symbols already processed. We retain at most $K = 5$ symbols, so $|h| = \min(K, t-1)$ before processing x_t. An order-k context s_k consists of the last k symbols of h, where $0 \le k \le |h|$. The order-0 context is the empty string ϵ. Thus, K is also the maximum context order. For each context s, a dictionary stores positive integer counts $c_s(x)$ for the symbols that have been inserted into that context. Define

$$\mathcal{A}_s = \{x : c_s(x) > 0\}, \qquad U_s = |\mathcal{A}_s|. \tag{1}$$

An absent context has an empty table. Contexts are stored as byte-string keys mapping to symbol-to-count dictionaries.

Both encoder and decoder start with empty dictionaries and empty history. They construct each prediction table before updating the PPM model with the current symbol. A *prediction table* lists the available symbols x and their integer weights $w_s(x)$ for context s. These weights are computed from the stored counts and determine the probability estimates used by the arithmetic coder. The update rule is described in Section 2.3.

2.2 Context Backoff and Symbol Exclusion

To construct prediction tables, we first search the available contexts. The PPM model visits $s_{|h|}, s_{|h|-1}, \ldots, s_0$ from longest to shortest.

For each new target symbol, initialize an exclusion set $E = \varnothing$. At the current order-k context $s = s_k$, the available candidates are

$$\mathcal{V}_s = \mathcal{A}_s \setminus E. \tag{2}$$

Here, $\setminus$ denotes set difference: remove all symbols in E from $\mathcal{A}_s$. If the target belongs to $\mathcal{V}_s$, encode it using the current context's prediction table and finish the search.

If $\mathcal{V}_s$ is empty, skip the context without coding an event. This covers both missing contexts and tables whose symbols have all been excluded.

If $\mathcal{V}_s$ is nonempty but does not contain the target, encode a special escape event ESC. Each nonempty prediction table contains both $\mathcal{V}_s$ and ESC. Internally, ESC is represented by integer 256. It produces no output symbol. After encoding ESC, set $E \leftarrow E \cup \mathcal{A}_s$ and continue to the next shorter context. This backoff mechanism allows the model to encode symbols not yet observed in the current context [1, 2].

Excluding higher-order predictions from lower-order tables is called *symbol exclusion* [2]. After an escape, these symbols cannot be the target, so their probability mass can be removed from subsequent symbol predictions. Symbol exclusion changes the prediction table without modifying the stored counts.

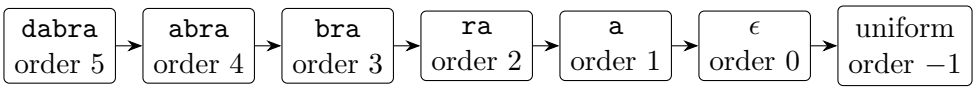


Figure 1: Search order for history `dabra`. Follow an arrow after an escape or an empty candidate set; stop when the target symbol is encoded. Each shorter nonnegative-order context removes the preceding context's first symbol.

If all nonnegative orders fail to find the target, use the order--1 table. It assigns integer weight 1 to every symbol in $\mathcal{B} \setminus E$, with no escape event:

$$P_{-1}(x \mid E) = \frac{1}{256 - |E|}, \qquad x \in \mathcal{B} \setminus E. \tag{3}$$

Thus a previously unseen symbol can always be encoded. In particular, the first input symbol is encoded using the uniform distribution over all 256 symbols.

2.3 Model Update

After the arithmetic coder encodes or decodes the input symbol x_t, visit the contexts from longest to shortest. If a context s does not contain x_t, set $c_s(x_t) = 1$ and continue to the next shorter context. Create an empty table first if the context is missing. At the first context that already contains x_t, increment its count by 1 and stop. All shorter contexts remain unchanged.

This rule is called *update exclusion* [2]. It determines which stored counts change, whereas symbol exclusion determines which symbols are available during prediction. Consequently, the stored counts need not equal substring occurrence counts.

Finally, append x_t to the history and retain only its last K symbols. Both counts and history are updated once per input symbol. No updates occur between successive escapes for the same symbol.

2.4 Tuned Integer Weights

Before symbol exclusion, standard PPM-D assigns integer weights $2c_s(x) - 1$ to known symbols and U_s to escape [3]. Inspired by this estimator, we instead use:

$$w_s(x) = 200c_s(x) \quad (x \in \mathcal{V}_s), \qquad w_s(\text{ESC}) = 86U_s. \tag{4}$$

The constants 200 and 86 were chosen experimentally for this implementation; their selection is evaluated in Section 5.

For each nonempty candidate set, the normalizing total and probabilities are

$$T_s(E) = 200 \sum_{x \in \mathcal{V}_s} c_s(x) + 86U_s, \tag{5}$$

$$P_s(x \mid E) = \frac{200c_s(x)}{T_s(E)}, \qquad P_s(\text{ESC} \mid E) = \frac{86U_s}{T_s(E)}. \tag{6}$$

Importantly, U_s counts *all* distinct symbols stored in the context, including excluded ones. Only the symbol term in $T_s(E)$ uses the reduced candidate set. The implementation does not replace U_s with $|\mathcal{V}_s|$.

Available symbols are ordered by increasing numeric value, followed by ESC; the order--1 table uses the same symbol ordering. The arithmetic coder uses the weights and cumulative sums directly. The decoder reconstructs the same tables and updates only after recovering a symbol, so frequency tables are not transmitted.

2.5 Worked Example

Consider encoding a space after `abracadabra`. Table 1 shows the state produced by the actual order-5 implementation with update exclusion. Symbols are displayed as ASCII characters. The history is `dabra`, giving the search order in Figure 1.

Order	Context	Stored counts	Excluded E	Available weights	Action
5	`dabra`	none	$\varnothing$	none	skip
4	`abra`	$c:1$	$\varnothing$	$c:200,\ \mathrm{ESC}:86$	ESC
3	`bra`	$c:1$	$\{c\}$	none	skip
2	`ra`	$c:1$	$\{c\}$	none	skip
1	`a`	$b:2,\ c:1,\ d:1$	$\{c\}$	$b:400,\ d:200,$ $\mathrm{ESC}:258$	ESC
0	ϵ	$a:4,\ b:1,\ c:1,$ $d:1,\ r:1$	$\{b,c,d\}$	$a:800,\ r:200,$ $\mathrm{ESC}:430$	ESC
-1	default	not stored	$\{a,b,c,d,r\}$	251 remaining symbols, each with weight 1	space

Table 1: Model trace for a space after `abracadabra`. Exclusion sets are shown on entry to each row; skipped contexts produce no coding event.

Order 5 is missing. The order-4 escape excludes `c`, leaving no candidates at orders 3 and 2. At order 1, the remaining symbol weights are 400 and 200, but the escape weight is $86 \times 3 = 258$: the excluded `c` still contributes to U_s. The next escape also excludes `b` and `d`. Order 0 retains only `a` and `r`, while its escape weight uses all five stored symbols.

The event sequence supplied to the arithmetic coder is therefore

$$\mathrm{ESC}_4, \quad \mathrm{ESC}_1, \quad \mathrm{ESC}_0, \quad \mathrm{space}_{-1}, \tag{7}$$

where subscripts indicate context order. Its model probability, before integer interval rounding, is

$$P(\text{space} \mid \texttt{abracadabra}) = \frac{86}{286}\,\frac{258}{858}\,\frac{430}{1430}\,\frac{1}{251}. \tag{8}$$

Only after this complete sequence do encoder and decoder update their counts and history with the space. This illustrates how a single input symbol may require several coding events while still producing exactly one output symbol.

3 Integer Arithmetic Coding

The PPM model supplies a prediction table for each coding event. The arithmetic coder uses its integer weights to narrow a numeric interval. As the interval narrows, the coder outputs bits that identify the selected interval. We use the integer coding framework described by Witten, Neal, and Cleary [4] and Howard and Vitter [3]. The rules below specify our implementation, including its 64-bit state and bitstream termination.

3.1 Prediction Tables and Coding State

Let $e_0, \ldots, e_{m-1}$ be the m entries of the current prediction table, in encoding order. Each entry is an available input symbol or ESC. Let w_j denote the integer weight of entry e_j. For a nonnegative-order context s, this is $w_s(e_j)$ from Equation (4). At order -1, every entry has weight 1. Define the cumulative weights and total weight as

$$C_0 = 0, \qquad C_j = \sum_{i=0}^{j-1} w_i \quad (1 \le j \le m), \qquad T = C_m. \tag{9}$$

Thus, entry e_j has cumulative range $[C_j, C_{j+1})$ and probability estimate w_j/T. The ordering is the one defined in the preceding section: increasing symbol values, followed by ESC when present.

The coder maintains an integer interval $[L, H]$, including both endpoints. We define

$$F = 2^{64}, \qquad M = F/2, \qquad Q = F/4, \qquad L = 0, \quad H = F - 1. \tag{10}$$

Here, F is the size of the full range, M its midpoint, and Q one quarter of its size. The interval is initialized once per file. It is carried across all coding events, including escapes and changes of context. Python integers allow intermediate widths and products to exceed 64 bits without overflow. All divisions below use integer arithmetic.

3.2 Interval Update

To encode entry e_j, first compute the current interval width $R = H - L + 1$. Then select its subinterval:

$$L' = L + \left\lfloor \frac{RC_j}{T} \right\rfloor, \tag{11}$$

$$H' = L + \left\lfloor \frac{RC_{j+1}}{T} \right\rfloor - 1. \tag{12}$$

Both expressions use the old value of L. Replace (L, H) with (L', H') only after computing both endpoints. The floor operation $\lfloor z \rfloor$ rounds z down to an integer. Larger weights receive wider subintervals, up to integer rounding.

3.3 Renormalization and Bit Output

After an interval update, the coder checks the three cases in Table 2. This step is called *renormalization*. It lets the coder output known bits while keeping the interval within the fixed range. Figure 2 shows the positions of the three cases.

Some bits must wait until a preceding bit is known. Let p count these *pending bits*, with $p = 0$ initially. Define emit(b) for a bit $b \in \{0, 1\}$ as follows: output b, then output p copies of $1 - b$, and reset p to zero. For example, if $p = 0$, then emit(0) outputs `0`; if $p = 2$, then emit(0) outputs `011`.

Case	Interval condition	Offset δ	Encoder action
E1	$H < M$	0	emit(0)
E2	$L \geq M$	M	emit(1)
E3	$Q \leq L < M \leq H < 3Q$	Q	Increase p by 1; output no bit yet

Table 2: Renormalization rules. Subtract the offset δ from each endpoint before shifting left by one bit. Apply the rules repeatedly after every coding event.

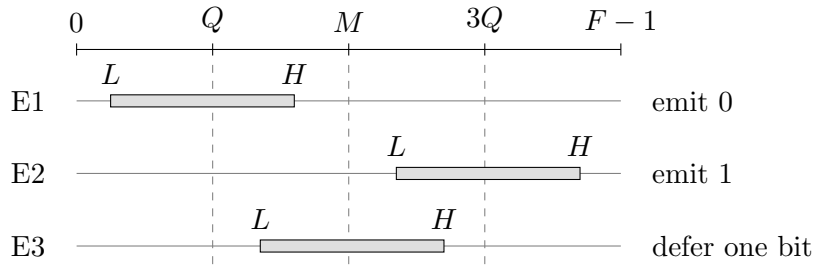


Figure 2: Schematic intervals before renormalization. E1 lies in the lower half and E2 in the upper half. E3 straddles the midpoint but lies within the middle half. Positions are illustrative.

For the selected case, perform its encoder action. Let $z \ll 1$ denote a one-bit left shift of a nonnegative integer z. Update the endpoints as follows:

$$L \leftarrow (L - \delta) \ll 1, \qquad H \leftarrow \big((H - \delta) \ll 1\big) + 1. \tag{13}$$

The left shift appends a 0 to L. For H, the additional $+1$ appends a 1 instead, preserving the inclusive upper endpoint. Check the conditions again using the new endpoints. Stop renormalizing when none applies, then process the next coding event. In E1 and E2, the interval determines the next output bit. In E3, that bit is still uncertain. Each E3 step records a following bit that will be the complement of the eventual output bit. The pending count is retained across coding events until an emit operation resolves it.

3.4 Final Bit Emission

After all input symbols have been encoded, the encoder must finish the remaining interval. For nonempty input, first increase p by 1. Then apply

$$\begin{cases} \text{emit}(0), & L < Q, \\ \text{emit}(1), & L \geq Q. \end{cases} \tag{14}$$

This final emission selects a bit prefix for the remaining interval and resolves the pending bits.

3.5 Arithmetic Decoding

The arithmetic decoder starts with the same interval $[0, F-1]$. It also maintains a *code value* V. Initialize $V = 0$, then read the first 64 bits of the payload in order. For each bit b, set $V \leftarrow (V \ll 1) + b$. Payload bit order and zero-padding are specified in Section 4.2.

For the current prediction table, compute its cumulative weights and $R = H - L + 1$. Map V into the cumulative-weight range using

$$u = \left\lfloor \frac{(V - L + 1)T - 1}{R} \right\rfloor. \tag{15}$$

Choose the unique entry e_j satisfying $C_j \leq u < C_{j+1}$. Use that entry to update the interval with Equations (11)–(12). The decoder then applies the same renormalization cases as the encoder. For each case, use its offset δ in Equation (13) and update the code value as

$$V \leftarrow \big((V - \delta) \ll 1\big) + b, \tag{16}$$

where b is the next payload bit. Repeat until no case applies. The decoder reads bits rather than emitting them; it does not maintain a pending-bit count. The recovered event e_j is returned to the PPM model. The file-level loop that requests these events is described next.

4 Compressed File Format and Reconstruction

The compressed file combines an original-length header with an arithmetic-coded payload. The header tells the decoder how many input symbols to recover. The decoder is configured with the same maximum order $K = 5$, 64-bit arithmetic state, and integer weights as the encoder. Both sides reconstruct the context statistics from previously processed symbols.

4.1 File Layout

Let N be the number of symbols in the original input. Each symbol corresponds to one byte, so N is also the original file size in bytes. Table 3 specifies the compressed file layout. Byte offsets start at zero.

Byte offset	Field	Representation
0–3	Original length N	Unsigned 32-bit integer, little-endian
4 onward	Arithmetic-coded payload	Bits packed most significant bit first

Table 3: Compressed file layout. The header occupies exactly four bytes. The payload size depends on the input and the model's predictions.

Little-endian means that the least significant header byte is stored first. If the four header bytes have values d_0, d_1, d_2, d_3, the decoder obtains

$$N = d_0 + 256d_1 + 256^2 d_2 + 256^3 d_3. \tag{17}$$

This field can represent $0 \leq N \leq 2^{32} - 1$. The arithmetic payload starts immediately after these four bytes. The decoder uses its fixed parameters and reconstructs the prediction tables; the file stores only the length and payload.

4.2 Bit Packing and Stopping Rule

The encoder places the first payload bit in the most significant position of the first payload byte. It fills each byte from bit 7 down to bit 0 before starting the next byte. If the final byte is incomplete, its remaining low bits are filled with zeros. For example, three remaining bits `101` are stored as `10100000`.

The decoder reads payload bits in the same order. It starts reading at byte offset 4, after the header. A request beyond the end of the stored payload returns a zero bit. This rule also applies when initializing the 64-bit code value V from a short payload.

The decoder stops after recovering exactly N input symbols. Escapes produce no output symbol and do not advance this count. Padding bits therefore do not create additional output symbols. Empty input is represented by the four zero header bytes and no payload. Its reconstruction is an empty output file. No end-of-file event is encoded.

4.3 Complete Encoding and Decoding Procedures

Algorithms 1 and 2 connect the PPM model, arithmetic coder, and file format. The following operations refer to the rules already defined:

- PredictionTables(h) supplies tables from longest context to shortest, with symbol exclusion. Each call starts with $E = \varnothing$. Empty candidate sets are skipped. Moving past a nonempty context adds its stored symbols to E. The final table is the uniform order--1 table over the remaining symbols.

- EncodeEvent($e, \mathcal{T}$) encodes event e using prediction table $\mathcal{T}$. It computes cumulative weights, updates the interval, and performs encoder renormalization.

- DecodeEvent($\mathcal{T}$) recovers one event from $\mathcal{T}$ using Equation (15). It updates the interval and performs decoder renormalization before returning the event.

- UpdateModel(x_t) applies the count and history updates in Section 2.3.

The table loop advances only after an escape. It ends as soon as an input symbol is encoded or recovered. Arithmetic state is retained across all tables and input symbols. The PPM model is updated only after a complete input symbol has been processed.

The encoder reads the original file in binary mode and saves the returned bytes as `compressed`. The decoder reads `compressed` in binary mode and writes the recovered symbols as bytes to `out.txt`. Identical table ordering, integer weights, and update rules keep both sides synchronized. The length header determines when reconstruction is complete.

Algorithm 1 Complete file encoding

Require: Input symbols $x_1, \ldots, x_N$
Ensure: Contents of the file `compressed`

header $\leftarrow$ four-byte little-endian representation of N
if $N = 0$ **then**
    return *header*
end if
Initialize an empty PPM model and history h
Initialize $L \leftarrow 0$, $H \leftarrow F - 1$, $p \leftarrow 0$, and an empty bitstream
for $t = 1, \ldots, N$ **do**
    for all prediction tables $\mathcal{T}$ in PredictionTables(h) **do**
        if x_t is an available symbol in $\mathcal{T}$ **then**
            $e \leftarrow x_t$
        else
            $e \leftarrow$ ESC
        end if
        EncodeEvent($e, \mathcal{T}$)
        if $e \neq$ ESC **then**
            break the table loop
        end if
    end for
    UpdateModel(x_t)
end for
Emit the final bits using Section 3.4
payload $\leftarrow$ pack the bitstream into bytes, padding with zeros
return *header* followed by *payload*

Algorithm 2 Complete file decoding

Require: Contents of the file `compressed`
Ensure: Reconstructed symbols for the file `out.txt`

Read N from the first four bytes as a little-endian integer
output $\leftarrow$ an empty symbol sequence
if $N = 0$ **then**
    return *output*
end if
Initialize an empty PPM model and history h
Initialize $L \leftarrow 0$, $H \leftarrow F - 1$, and read V from the payload
    using Section 3.5 and zero-padding
for $t = 1, \ldots, N$ **do**
    for all prediction tables $\mathcal{T}$ in PredictionTables(h) **do**
        $e \leftarrow$ DecodeEvent($\mathcal{T}$)
        if $e \neq$ ESC **then**
            $x_t \leftarrow e$
            Append x_t to *output*
            break the table loop
        end if
    end for
    UpdateModel(x_t)
end for
return *output*

5 Experiments and Results

The experiments examine parameter selection, the contribution of individual design choices, and comparisons with other compression methods. All 20 distinct configurations completed three independent decoding runs, and every run reproduced the input byte for byte. The selected final configuration is reused across the four comparisons.

5.1 Experimental Setup and Evaluation Metrics

The input is the project file `austen.txt`, containing $N = 2{,}664{,}245$ bytes. Every method operates on the same binary input. Parameter selection also uses this file, so the reported choices are specific to this project input. They have not been selected using a separate test set.

Let C be the size of `compressed`, including its header and any stored model information. Let D be the compiled decoder's `.pyc` size. The main evaluation metric is

$$R_{\text{total}} = \frac{C + D}{N} \times 100\%. \tag{18}$$

A smaller value is better. Reporting C and D separately distinguishes improved compression from a smaller decoder.

The validation script minifies each decoder with python-minifier, renaming global identifiers, removing literal statements, and preferring single-line output. The minified source is compiled with optimization level 0 and the embedded filename `decompress.py`. Each decoding run executes this same compiled decoder in an isolated working directory and compares `out.txt` with the original binary input. Source snapshots, artifact hashes, and individual timing logs are retained with the experimental results.

The reported decoding time is the median user CPU time of three independent runs, measured with `/usr/bin/time -p`. Encoding, minification, and compilation are excluded from this time. All configurations were measured sequentially under the environment in Table 4. These are local measurements; execution time on the ECE Linux cluster has not been measured in this benchmark.

Environment field	Value
Operating system	macOS 26.7.1, arm64
CPU	Apple M3; 8 logical CPUs
Exact Python version	CPython 3.12.4, built with Clang 18.1.8
Python minifier version	python-minifier 3.2.0
Decoding repetitions	3 per configuration; median user CPU time reported

Table 4: Common local benchmark environment, recorded on October 29, 2026.

5.2 Parameter Selection

5.2.1 Maximum Context Order

The maximum context order determines how much preceding input is available for prediction. A longer context can make more specific predictions, but its statistics may be sparse. Increasing the order therefore need not improve compression [2].

Table 5 compares $K \in \{4, 5, 6\}$ using the final weights $200c_s(x)$ and $86U_s$, update exclusion, the same 64-bit arithmetic coder, and the four-byte header. Order 5 gives the smallest file among these candidates: 23,411 bytes smaller than order 4 and 3,648 bytes smaller than order 6. This supports retaining $K = 5$ for this input; it does not establish an optimum over all orders or other data sets.

Maximum order K	Compressed size C (bytes)
4	640,200
5 (selected)	616,789
6	620,437

Table 5: Context-order comparison with the final weights and common compact file format. The decoder size is 4,934 bytes for all three configurations.

5.2.2 Relative Symbol and Escape Weights

The final probability estimator uses

$$w_s(x) = \alpha c_s(x), \qquad w_s(\mathrm{ESC}) = \beta U_s. \tag{19}$$

Here, α controls the relative weight of observed symbols and β controls escape. Their ratio $\rho = \beta/\alpha$ determines the balance. For fixed counts and candidates, reducing ρ assigns less probability to escape and more to the available symbols.

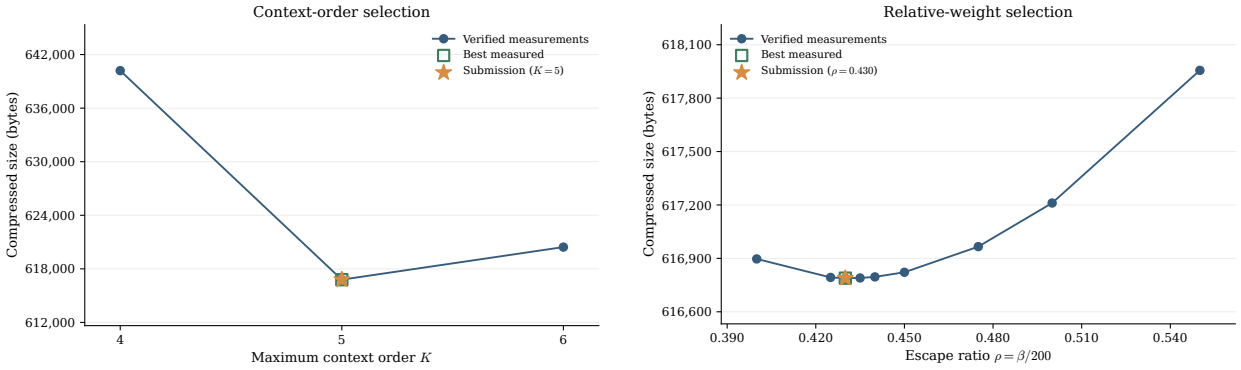
Table 6 fixes $\alpha = 200$ and $K = 5$ while varying β. All candidates use update exclusion, the same symbol ordering, the same arithmetic coder, and the four-byte header. The smallest measured file uses

$$\alpha = 200, \qquad \beta = 86, \qquad \rho = 0.43. \tag{20}$$

Reducing β from 110 to 86 saves 1,167 bytes. Further reductions to 85 and 80 increase the file size. The minimum is shallow: $\beta = 86$ saves only one byte relative to 87 and four bytes relative to 85. Thus 86 is the best tested value on this input, with little practical advantage over its immediate neighbors. It is not claimed to be a global optimum over all weight rules.

Escape factor β	Ratio $\rho = \beta/200$	Compressed size C (bytes)
110	0.550	617,956
100	0.500	617,211
95	0.475	616,966
90	0.450	616,822
88	0.440	616,796
87	0.435	616,790
86 (selected)	0.430	616,789
85	0.425	616,793
80	0.400	616,897

Table 6: Escape-weight comparison with fixed symbol factor $\alpha = 200$. The decoder size is 4,934 bytes for every candidate.



(a) Maximum context order.

(b) Relative escape weight.

Figure 3: Parameter comparisons for the final model. Each panel varies one parameter while holding the other fixed; the tested minima are $K = 5$ and $\beta/200 = 0.43$.

5.3 Effect of Individual Design Choices

Table 7 separates the changes made during development. All configurations use $K = 5$, symbol exclusion, the same arithmetic coder, and the same symbol ordering. The first five rows use a common 13-byte header and a common decoder template; the last row uses the compact format and specialized decoder. Every decoder is minified and compiled using the same procedure.

The PPM-C baseline uses symbol weights $c_s(x)$ and escape weight U_s. The standard PPM-D row uses $2c_s(x) - 1$ and U_s. The next row replaces the symbol weight with $2c_s(x)$. The first three configurations update every context. The following rows add update exclusion, then the tuned weights $200c_s(x)$ and $86U_s$. This sequence separates the effect of the estimator from the effect of the count-update rule.

Configuration	Compressed bytes C	Decoder bytes D	R_{total}
PPM-C	629,261	5,317	23.8183%
Standard PPM-D	623,758	5,342	23.6127%
Modified symbol weight $2c_s(x)$	619,582	5,332	23.4556%
Modified weights + UE	617,220	5,327	23.3667%
UE + tuned weights	616,798	5,342	23.3515%
Tuned model + compact header and decoder	616,789	4,934	23.3358%

Table 7: Design comparison with matched configurations and a common compilation procedure. UE denotes update exclusion.

Every step in this sequence reduces the total size on this input. Changing PPM-C to standard PPM-D reduces C by 5,503 bytes; replacing $2c_s(x) - 1$ with $2c_s(x)$ saves a further 4,176 bytes. Adding update exclusion then saves 2,362 compressed bytes, and tuning the relative weights saves another 422 compressed bytes. The corresponding total-size savings also include the decoder-size changes shown in the table; they need not equal these payload savings.

The final compaction step reduces C by exactly 9 bytes and D by 408 bytes, saving 417 bytes in total. The arithmetic-coded payloads of the tuned legacy and compact formats are byte-identical, confirming that this final step changes format and decoder overhead rather than prediction quality. Both decoders were already minified; the additional decoder saving comes from the compact implementation. Relative to PPM-C, the final total size falls from 634,578 to 621,723 bytes, reducing R_{total} from 23.8183% to 23.3358%.

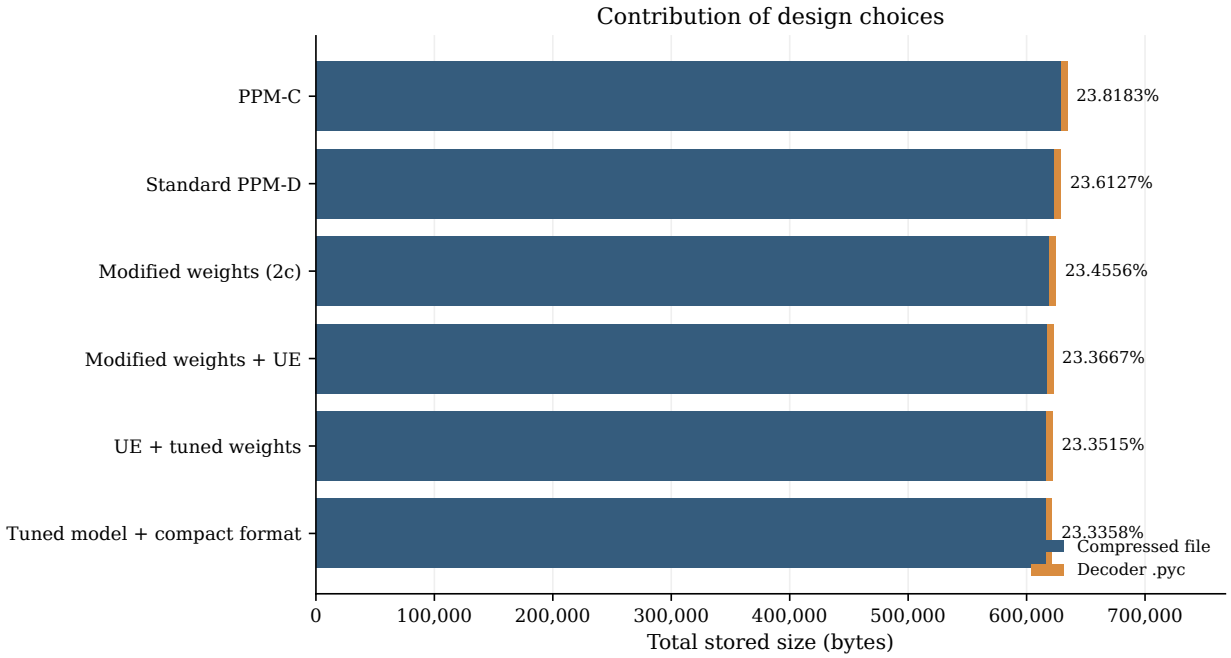


Figure 4: Compressed-file and compiled-decoder contributions to total stored size. Labels give R_{total}; exact values appear in Table 7.

5.4 Comparison with Other Compression Algorithms

The comparison includes the project's implementations of Huffman coding [5], context-free arithmetic coding [4], LZ78 [6], and LZW [7]. Each method uses its own complete file format. The value C therefore includes any stored Huffman tree, frequency table, or other decoding information required by that method.

The Huffman implementation groups two consecutive bytes into a symbol and stores its decoding tree. A final short group is retained if the input length is odd. The context-free arithmetic baseline uses one frequency table computed from the complete input and stored in the file. The LZ78 implementation stores phrase-index and literal pairs. The LZW implementation stores phrase codes. Both dictionary implementations choose a fixed, whole-byte index width for the file. These details affect their compressed sizes and are part of the implementations being compared.

The final tuned PPM method has the smallest total size among these implementations: 621,723 bytes, or 23.3358% of the input. LZW is the next smallest at 1,248,775 bytes (46.8716%); the final method uses 50.21% less total storage than this LZW baseline. Its median decoding time is 7.73 seconds, with a measured range of 7.62–7.91 seconds, making it slower than all four comparison methods. The size advantage therefore comes with a decoding-time trade-off.

Method	Compressed bytes C	Decoder bytes D	R_{total}	Decode CPU time (s)
Huffman (two-byte groups)	1,308,812	3,300	49.2489%	0.49
Context-free arithmetic coding*	1,444,892	2,844	54.3394%	4.28
LZ78	1,475,432	1,414	55.4321%	0.10
LZW	1,247,197	1,578	46.8716%	0.15
Final tuned PPM + UE + arithmetic	616,789	4,934	23.3358%	7.73

Table 8: Compression-method comparison on `austen.txt`. Times are medians of three user CPU measurements. The starred reference implementation imports `bisect` in addition to `struct`, exceeding the assignment's permitted imports; it is not a submission candidate.

The context-free arithmetic baseline uses 54.3394% of the input size, compared with 23.3358% for the final model. This comparison supports the benefit of context modeling in these implementations, although their file formats and probability models also differ. The Huffman and dictionary baselines provide comparisons with other coding approaches. These results apply to the tested implementations and this input, rather than establishing a general ranking of the algorithm families.

All measured local decoding times are below 1,200 user CPU seconds. The final decoder imports no modules. These observations do not replace validation on the required ECE Linux host.

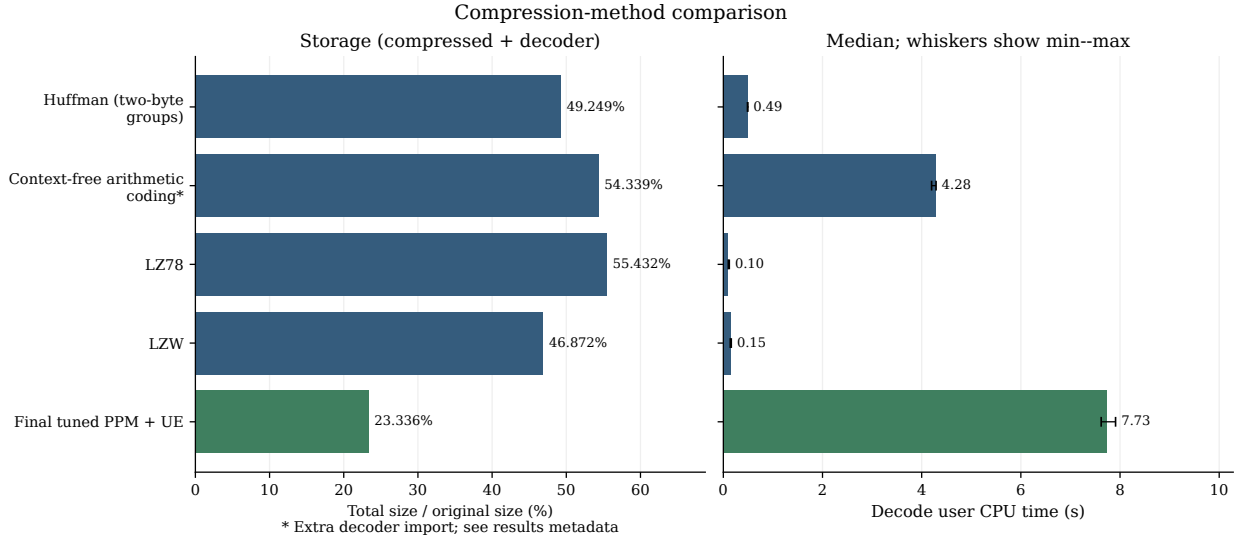


Figure 5: Total storage and decoding user CPU time. Timing whiskers show the minimum and maximum of three runs. The starred arithmetic implementation is a reference baseline with an extra import, as described in Table 8.

6 CONCLUSIONS

On the project input, the selected order-5 PPM model with update exclusion, weights $200c_s(x)$ and $86U_s$, and 64-bit integer arithmetic coding produces a 616,789-byte compressed file and a 4,934-byte compiled decoder. Their combined size is 621,723 bytes, or 23.3358% of the original. All three measured decoding runs reconstruct the input exactly, with a median local user CPU time of 7.73 seconds. The controlled comparisons support the chosen parameters and each successive size reduction. The final method uses the least storage among the tested implementations but decodes more slowly. These conclusions are specific to this input and local environment; ECE Linux runtime validation remains separate.

References

[1] J. G. Cleary and I. H. Witten, "Data compression using adaptive coding and partial string matching," *IEEE Transactions on Communications*, vol. 32, no. 4, pp. 396–402, 1984. doi:10.1109/TCOM.1984.1096090.

[2] W. J. Teahan, "Probability estimation for PPM," in *Proceedings of the New Zealand Computer Science Research Students Conference*, University of Waikato, Hamilton, New Zealand, pp. 267–274, 1995. Online text.

[3] P. G. Howard and J. S. Vitter, "Practical implementations of arithmetic coding," in *Image and Text Compression*, J. A. Storer, Ed. Kluwer Academic Publishers, 1992, pp. 85–112. Technical report version.

[4] I. H. Witten, R. M. Neal, and J. G. Cleary, "Arithmetic coding for data compression," *Communications of the ACM*, vol. 30, no. 6, pp. 520–540, 1987. doi:10.1145/214762.214771.

[5] D. A. Huffman, "A method for the construction of minimum-redundancy codes," *Proceedings of the IRE*, vol. 40, no. 9, pp. 1098–1101, 1952. doi:10.1109/JRPROC.1952.273898.

[6] J. Ziv and A. Lempel, "Compression of individual sequences via variable-rate coding," *IEEE Transactions on Information Theory*, vol. 24, no. 5, pp. 530–536, 1978. doi:10.1109/TIT.1978.1055934.

[7] T. A. Welch, "A technique for high-performance data compression," *Computer*, vol. 17, no. 6, pp. 8–19, 1984. doi:10.1109/MC.1984.1659158.